At travel_start_date, group becomes IMMEDIATELY read-only for group details, RSVP, and member management. No edits, no new invitees. Forum chat stays open separately for in-trip and post-trip discussion.

18.11 Coordinator Portal Tabs

Tab	Purpose
Overview	Summary stats, recent activity, group rate countdown
Invitees	List with status, drill-in to each
Edit Group	Details, hero image, message, anonymity
Preview Email	Live invitation email preview
Forum	Forum chat moderation (also see §19)

Section 19 — Forum-Style Group Chat

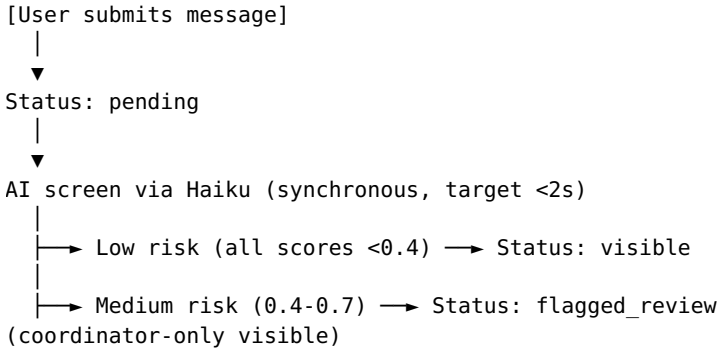
19.1 Model

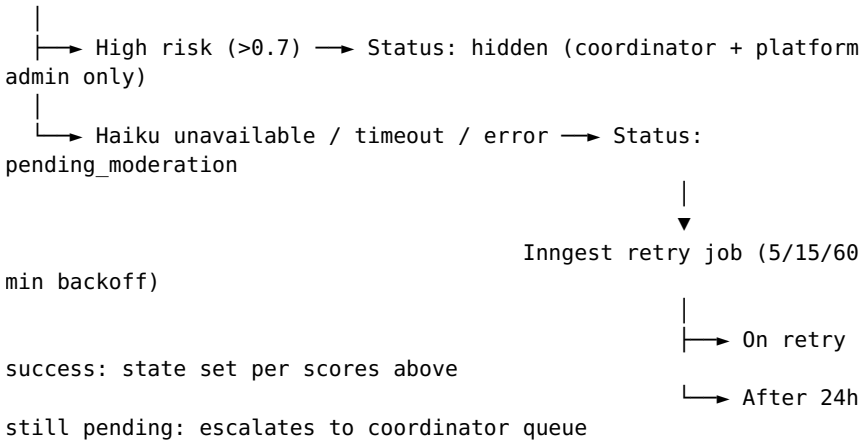
Asynchronous threaded forum, not real-time chat. Better fit for cruise planning that spans months. Each group has one forum, with threads and one-level replies.

19.2 Posting Permissions

- Invitees: create threads, reply, react, edit/delete own messages
- Coordinator: invitee permissions + lock threads, hide messages, mute users, post announcements
- Muted users: read only
- RSVP not_going: read only
- Forum-locked group: only coordinator can post

19.3 AI Moderation Pipeline





Failure modes and behavior

The contract: **fail-closed by default, distinct from coordinator-review states, with automatic recovery.**

Failure	Result	Customer-visible?	Recovery
Haiku timeout (>2s)	Status: `pending_moderation`	No (not yet visible)	Inngest retry job re-screens on backoff (5/15/60 min)
Haiku API error	Status: `pending_moderation`	No	Same retry job
Haiku returns malformed response	Status: `pending_moderation` + flagged for engineering attention	No	Same retry job; engineering investigates malformed responses
Haiku auth failure / quota exhaustion	Status: `pending_moderation` + alerts platform admin	No	Manual intervention
Retry job fails for >24h	Status: `flagged_review`	No initially; then coordinator-visible if escalates	Coordinator handles via existing flow

Why fail-closed (not fail-open)

Fail-open posts every message immediately when Haiku is unavailable. Under sustained Haiku unavailability or a sophisticated attack timing message posts during Haiku outages, this creates an abuse window. The cost of fail-closed (a brief invisible-message state while retry runs) is far smaller than the cost of an abuse incident in a group forum.

Why a separate `pending_moderation` state (distinct from `flagged_review`)

`flagged_review` is for messages that Haiku *successfully* screened and flagged as medium-risk. They require coordinator judgment. `pending_moderation` is for messages that haven't been screened yet — fundamentally different from a coordinator's perspective. Mixing the two would either waste coordinator time auto-screening Haiku-recovered messages, or hide engineering-side outages behind apparent moderation queue depth.

The retry job

`forum-moderation-retry` Inngest function:

- Triggers on any message entering `pending_moderation` status.
- Initial backoff: 5 minutes. Second retry: 15 minutes. Third retry: 60 minutes.
- Each attempt re-runs the full Haiku screen.
- On success: message status is set per the screening scores.
- After 24 hours total elapsed time without successful screening: status moves to `flagged_review` with reason `"moderation_timeout"`. Coordinator handles manually.
- Each retry attempt writes to `audit_log` with the attempt count and Haiku-response detail.

The retry job uses `tenantClient` with `tenantContextFromInngestEvent` per the §11.2.2 pattern. The event payload includes the message's `tenant_id` and `forum_id`.

Schema addition

```
        ALTER TABLE public.forum_messages
          ADD COLUMN pending_moderation_since TIMESTAMPTZ,
          ADD COLUMN moderation_attempt_count INTEGER NOT NULL DEFAULT 0,
          ADD COLUMN moderation_last_attempt_at TIMESTAMPTZ,
          ADD COLUMN moderation_last_error TEXT;

        -- Add 'pending_moderation' to the forum_messages.status CHECK
constraint.
```

What the user sees

A user who submits a message in a forum:

- Normal path: message appears immediately (status `visible`) or quietly waits (status `pending_moderation` looks identical to `pending` to the submitter).
- Retry succeeds: message becomes visible to others.
- Retry fails 24h: submitter sees "Your message is awaiting review" — same UX as `flagged_review`.

Calls Worth Flagging

- **Fail-closed is non-negotiable for forums.** Fail-open here is a screenshot-able PR incident waiting to happen.
- **The retry job MUST be idempotent.** Use optimistic locking on `moderation_attempt_count`.
- **24h timeout to coordinator escalation is generous.** Tune downward if coordinator queue noise from sustained Haiku issues becomes the bigger problem.
- **Haiku quota exhaustion alerts the platform admin, not the**

coordinator. Quota exhaustion is a platform-level issue, not a tenant-level one.

19.4 Screening Dimensions

- Spam (off-platform commerce links, repetition, low-content)
- Abuse (insults, slurs, threats, harassment)
- PII leak (other people's contact info, credit cards — zero tolerance for credit cards)
- Off-topic promotional content
- Trip misinformation
- Solicitation (selling other travel services)
- Prompt injection attempts

19.5 Reactions

Bounded emoji set:

- 👍 thumbs up
- ❤ heart
- 😂 laugh
- 😮 surprised
- 🎉 celebrate
- 👀 eyes / interesting

19.6 Anonymity in Forum

- Display name shown as 'Anonymous' or 'Anonymous Traveler' instead of real name
- Avatar replaced with neutral placeholder
- user_id still stored for moderation; UI never reveals identity
- Coordinator and platform admin always see real names for moderation

19.7 Coordinator Tools

- Per-message: hide / unhide / pin within thread
- Per-user: mute (temporary or indefinite) / unmute / view recent posts
- Per-thread: lock / pin / mark announcement (read-only) / delete (soft)
- Forum-wide: lock entire forum / unlock

19.8 Edit and Delete

- Edit own message: any time, no expiry
- ‘Edited’ indicator shown with last-edited timestamp
- Edit history preserved in JSONB for audit
- Delete own message: soft delete; ‘[message deleted]’ placeholder if replies exist

19.9 User Strike System

- 3 AI-hidden messages in 24h → automatic 24-hour mute
- 5 coordinator-hidden messages → coordinator review prompt
- 10 cumulative strikes → recommend coordinator consider removing invitee

Recommendations, not automatic bans. Coordinator has final say.

19.10 Post-Sailing Forum

Forum stays fully active for in-trip and post-trip conversation. Coordinator can post updates, photos, post-trip thanks. AI screening continues. No automatic forum closure — coordinator can manually lock when group has run its course.

19.11 Photo Support — Deferred

v6 launch: text-only forum. Photos deferred to v7 due to storage, moderation, EXIF stripping, and mobile UX complexity. Users can share photo links to external hosting (Google Photos, iCloud share).

Section 20 — Booking Flow

DESIGN STATUS: ** **DEFERRED for final design until host agency is selected. Many host agencies provide booking widgets (iframe or embedded JS) that handle the booking UI directly. v6 §20 draft documents the platform-native flow as a fallback. Once host is chosen, decision needed: use host widget vs build native flow.

20.1 Considerations Regardless of Implementation Path

- AI co-pilot, CRM integration, group context, persona handoff — platform-native regardless
- Commission tracking and reconciliation — platform-side
- Booking record (commissionable_fare, fare_breakdown) — must be captured per §14 even if host widget owns the form

- DOB-not-ages, estimated DOB confirmation requirements — must apply somewhere in flow
- Multi-traveler contact creation — handled platform-side after submission
- All §20 schema fields stay valid in either implementation path

20.2 Platform-Native Flow (Fallback Reference Design)

[Quote accepted] OR [Direct booking request]

|

▼

[Draft booking created]

|

▼

Stage 1: Confirm trip details

|

▼

Stage 2: Passenger details (DOBs not ages)

|

▼

Stage 3: Booking options (insurance, addons)

|

▼

Stage 4: Review and confirm (with disclosures)

|

▼

Submit to host adapter

|

▼

Booking confirmed / pending host review / rejected

|

▼

Confirmation page + email

20.3 Entry Points

Entry	Context
From a quote	Quote accepted → booking flow with quote data pre-filled
From AI chat	AI’s collect_booking_details tool → flow with conversation context
From a group invitation	Invitee clicks ‘Book my cabin’ → group context pre-filled
From ‘Book now’ CTA	Direct entry, blank booking
From price watch alert	Watch fires → ‘Book at this price’ → flow with watch context

20.4 AI Co-Pilot

Persistent panel beside the booking form. Modes:

- Passive (default): quiet; shows stage indicator, quick stats, suggestion chips
- Active: customer types or asks; chat panel expands

Co-pilot can do:

- Answer field-specific questions
- Compare options
- Pull from CRM with permission
- Validate input (typo detection)
- Detect concerns (tight final-payment timeline)
- Pre-fill from memory

Co-pilot cannot submit booking — only the form does that. All co-pilot responses go through supervisor preflight per §10.

20.5 DOB Confirmation Gate

If a passenger has date_of_birth_is_estimated = true (from age-only mention), field shows pre-filled with warning icon: ‘We have [name]’s DOB as approximately [date]. Please confirm or update.’ Booking cannot submit until all estimation flags cleared.

20.6 Validation Layers

Layer	When	Examples
Per-field	Real-time as customer types	Email format, DOB sanity, passport expiry > sailing date + 6 months
Cross-field	On stage transition	All passengers have DOBs; pricing math sums
		All required fields

Final submission	Before host adapter call	populated; cruise line age rules satisfied; promo codes verified

20.7 Sub-Host Tenant-of-Record Disclosure

Stage 4 (Review) explicitly shows the legal entity of record:

This booking will be made through [Host Agency Name]. Your sub-host: [Tenant Name]. Customer service contact: [Tenant Support Email].

Required for transparency. Customer sees who's actually selling them the trip even when site is white-labeled.

20.8 No Anonymous Bookings

Booking requires a signed-in account. Booking flow detects anonymous state and redirects to signup with the in-progress draft preserved (per §11.6 anonymous-to-authenticated transfer).

20.9 Cancellation & Modification

- Cancellation: shows impact (refund, lost deposits) → customer confirms → adapter.cancelBooking() → status to 'cancelled' → commission reversal per §14
- Modification: cabin change, passenger add/remove, insurance changes — varies by cruise line and proximity to sailing; handled via adapter.modifyBooking()

Section 21 — RAG Knowledge Base (Consumer Side)

How retrieved knowledge is used in chat conversations: filtering, prompt inclusion, citation, freshness handling, and the hallucination defense layer.

21.1 Retrieval Flow in Chat

[User message arrives]

|

▼

Extract entities (destinations, cruise lines, dates, party type, intent)

|

▼

Build query for RAG (message text + entities + persona specialty)

|

▼

Call RAG /retrieve (per §8.4)

|

▼

Filter and sort chunks (confidence floor, deduplication, top N)

|

▼

Format into RAG knowledge block

|

▼

Include in system prompt for Claude Sonnet

|

▼

Stream response

|

▼

Log retrieval feedback signals post-response

21.2 Entity Extraction

Lightweight Haiku call before RAG. Extracts:

- Destinations (Alaska, Glacier Bay)
- Cruise lines (Royal Caribbean)
- Ships (Symphony of the Seas)
- Travel dates
- Passenger composition (couple, family with kids)
- Intent (research, compare, book, support)
- Categories hint (cabin_intel, dining, ports, etc.)

21.3 Chunk Filtering Before Prompt Inclusion

- Confidence floor: chunks with composite_confidence < 0.35 dropped

- Expired chunks: excluded
- Closed-to-new promos: only included with contact-has-booking signal
- Similarity dedup: similar chunks (>0.8) — only highest scored kept
- Final cap: top 3-5 chunks (default 4) included in prompt

21.4 RAG Knowledge Block Format (in Prompt)

KNOWLEDGE CONTEXT (retrieved for this turn):

[1] Authority: 0.92 (official) | Recency: 0.85 | Match: 0.91 | Confidence: 0.89

Category: cabin_intel | Source: royalcaribbean.com

Indicator: ★★★★★ Verified official source

Royal Caribbean’s Symphony of the Seas Junior Suites feature [content…].

[2] Authority: 0.85 (tenant_verified) | Recency: 0.78 | Match: 0.84 | Confidence: 0.82

Category: dining | Source: tenant verified

Indicator: ★★★★ Verified by your agency

Symphony of the Seas dining venues include [content…].

INSTRUCTIONS:

- Higher-confidence chunks should be treated as more reliable.
- “Verified” chunks are explicitly approved; trust their facts.
- “Tenant knowledge” chunks may be more situational.
- Always include a freshness caveat for pricing or time-sensitive info.
- If chunks contradict each other, prefer the higher-authority chunk and note the discrepancy if relevant.
- If no chunks are highly relevant to the user’s question, say so honestly.

21.5 Citation in AI Responses

Inline source attribution lets customer evaluate trust:

Symphony of the Seas Junior Suites typically have a 220 sq ft layout with a private balcony, per Royal Caribbean’s official site. Pricing varies seasonally; let me look up current rates.

For sub-host scenarios, tenant-submitted content framed as ‘your agency’s records’ — preserves white-label trust without misrepresenting authority.

21.6 Customer-Facing Source Transparency

Chat UI shows subtle ‘source’ indicator on AI responses that used RAG. Hover/click expands to: source title, confidence tier (★ rating), excerpt. Tenant-configurable: tenants can disable source display for minimal interface.

21.7 Outdated Information Handling

Time-sensitive content auto-flagged for freshness caveats: pricing, promos, schedules, frequently-changing policies. Hard-expiry chunks excluded. UI shows ‘may be outdated’ badge for chunks older than category halflife.

21.8 Conflict Handling

Two chunks may disagree. AI prompted to:

- Prefer higher authority by default
- Weight recency too — newer often trumps stale official
- Surface conflict honestly when relevant
- Authoritative_override chunks win against any other

21.9 No-Result Handling

When RAG returns nothing useful (all chunks below 0.35), AI receives empty knowledge block with instructions:

- Don’t fabricate
- Answer from general knowledge with appropriate caveats if possible
- Acknowledge limitation: ‘I don’t have specific information on that. Would you like me to check with our team?’
- Consider escalation

21.10 §21.19 Hallucination Defense (Consolidated)

There is no single mechanism. The platform stacks eight layers of defense. The goal: be wrong rarely; when wrong, be wrong in low-consequence ways; when high-consequence, escalate rather than commit.

Eight-Layer Defense

Layer	What It Does	Catches
	Forbids unsupported	

1. Persona prompt instructions	facts, requires hedging, freshness caveats	Baseline; cheapest defense
2. RAG grounding	Provides retrieved chunks with authority/recency	Factual hallucination on topics with RAG coverage
3. Hallucination risk check	Supervisor extracts factual claims, checks against RAG	Fabricated facts; regenerates with strict grounding
4. Confidence floor	Chunks below 0.35 filtered out before AI sees them	Anchoring on weak content
5. Promise detection	Catches ‘guaranteed’, ‘I assure you’	Over-confident commitments
6. Arithmetic check	Validates any math in response	Made-up plausible numbers
7. Customer feedback	Thumbs-down feeds authority nudges	Pattern detection over time
8. Escalation safety net	Low-confidence topics route to human	When in doubt, get a human

Topic-Specific Guards

- Pricing: chunks auto-capped at 0.55; freshness caveat injected; arithmetic check enforced; **quote PDFs include “ESTIMATE” header and a “subject to confirmation at booking” footer in all cases where the host has not provided a price-lock guarantee per §21.10.1 below.**
- Medical/accessibility/dietary: persona forbids advice; defaults to escalation
- Legal/contractual: persona forbids advice; cancellation policy references documentation only
- Date and availability: never asserted by AI; always from search_host_inventory tool
- Loyalty programs: cited from RAG only; suggests member-direct verification

Known Residual Risk

- Subtle factual errors in narrative responses (may evade claim extractor)
- Wrong source attribution
- Plausible-but-outdated scheduling
- Multi-fact synthesis errors
- Memory-level hallucinations (claimed conversation context that didn’t happen)

Tuning Levers (Post-Launch)

- Raise confidence floor from 0.35 to 0.50
- Mandatory citation for all factual claims
- Require multi-chunk consensus for confident assertion
- Constrain training-knowledge fallback for high-stakes categories
- Pre-flight fact-check (second AI call to verify the first)

Measurement Points

- Hallucination risk check fire rate
- Block-after-regen rate
- Escalation rate due to confidence floor
- Thumbs-down rate per persona, per topic
- Per-chunk role in flagged responses (feeds authority nudge)
- Compound metric: thumbs-down × confidence (low confidence + high feedback = overreaching)

21.10.1 Quote Pricing Discipline

Quotes are estimates until the host confirms. The platform's quote PDF and quote acceptance flow reflects this explicitly to prevent "the platform promised me $X" disputes when the booking comes back at $Y.

When is a price "confirmed"?

A price is "confirmed" only if BOTH:

1. The host adapter returned the price within the last 15 minutes, and
2. The host adapter exposes a `price_lock_token` or equivalent guarantee mechanism, and the token is currently valid.

Without both conditions, the price is an "estimate." Most host adapters at launch will NOT expose price-lock — so default is estimate.

Quote PDF rendering

For ESTIMATE quotes (default):

- **Header:** "ESTIMATE — pricing subject to confirmation at booking"
- **Inline:** every price line shows a small "est." marker.
- **Footer:** "This quote is an estimate. Final pricing will be confirmed at booking submission and may differ. By accepting this quote, you authorize us to submit a booking at the host's then-current price within $X variance of the estimate above. Bookings beyond $X variance will be sent back to you for re-confirmation." where `X` is configurable per tenant (default $50).
- **Validity period:** 7 days. After 7 days, the quote auto-expires and the customer is asked to request a refresh.

For CONFIRMED quotes (rare at launch):

- **Header:** “QUOTE — price locked through [timestamp]”
- **Inline:** no “est.” marker.
- **Footer:** “Price locked by [host name] through [timestamp]. If booking is not submitted by this time, the price will be re-quoted.”
- **Validity period:** matches the price-lock token’s expiry.

Quote acceptance flow

When the customer accepts a quote:

- ESTIMATE acceptance: the platform records the estimate price and the customer’s authorized variance ($X default). Booking submission proceeds with the host. If the host returns a price within variance, booking proceeds. If outside variance, the booking is paused in `pending_customer_reconfirmation` status; customer is emailed with the new price and a one-click re-confirm action.
- CONFIRMED acceptance: the platform records the price-lock token and submits within its validity. If the token expires before submission, the quote is automatically re-quoted and the customer is asked to re-accept.

Booking-submit handler contract

`/api/bookings/:id/submit` extends from §7.4:

1. Resolve the quote underlying the booking.
2. If quote was CONFIRMED and price-lock token is still valid: submit at the locked price.
3. If quote was ESTIMATE:
   a. Call host adapter for current price.
   b. Compute variance against the accepted estimate.
   c. If within variance: submit at host’s current price.
   d. If outside variance: mark booking `pending_customer_reconfirmation`; notify customer; do NOT submit.

Audit

Every quote send, acceptance, and reconfirmation writes to `audit_log` with the rendered price snapshot. This protects against later disputes: “you told me $3,847” is answered by the audit record of what the customer saw and accepted.

Configuration

Setting	Default	Where
Default ESTIMATE variance allowed at submission	$50	`tenant_settings.quote_variance_cents` (per-tenant)
ESTIMATE quote validity period	7 days	`platform_settings.quote_estimate_validity_day`

Calls Worth Flagging

- **Default-to-estimate is the safe default.** Most hosts at launch won't expose price-lock. Treating every quote as an estimate protects the platform from over-commitment.
- **Variance threshold matters for customer experience.** $50 is the right rough number — small enough that customers don't get surprised, large enough that minor host-side price adjustments don't bounce every booking back. Tune per tenant if needed.
- **Audit snapshot at acceptance is the dispute defense.** If the customer later claims "you quoted me $3,847," the audit log shows what they accepted, with what variance, on what date. The dispute resolves quickly.

Section 22 — Knowledge Ingestion (RAG Submission Side)

How content enters the knowledge base. Covers submission methods, the normalization pipeline, the tenant review queue, and the revised global review model.

22.1 Submission Methods

Method	Use Case	Notes
Web UI single-item submit	Quick add from /knowledge/submit	Form with content, source URL, category hints
Browser extension	While browsing supplier sites	Right-click → 'Add to knowledge'; captures URL, title, selection
iOS Shortcut	Share-sheet from mobile Safari/Mail/Files	Send selection or page to platform
File upload	PDFs, docs, spreadsheets, decks, images	Up to 50MB per file; supported: PDF, DOCX, XLSX/XLS, PPTX/PPT, TXT, MD, HTML, JPG, PNG
Manual entry	Free-form text in the web UI	For knowledge that doesn't come from a source
Batch API	Programmatic bulk import	Up to 100 items per call; auth via service token

22.2 No Submission Limits

DESIGN DECISION: **Earlier drafts had daily submission limits and per-tenant chunk caps. These created a perverse incentive (tenants hoarding rather than contributing). Removed in v6. Spam and abuse handled via quality screening, not quotas.

rag_submit_daily_limit and rag_chunks_max in tenant_registry are nullable; null means unlimited. Abuse handled via §27 abuse monitoring (quality patterns, not volume).

22.3 Supported File Types

Type	Extension(s)	Handler
PDF	.pdf	pdf-parse + OCR fallback for scanned
Word	.docx, .doc	mammoth for docx; LibreOffice convert for legacy doc
Excel	.xlsx, .xls	SheetJS; one chunk per sheet typically, with structure preservation
PowerPoint	.pptx, .ppt	Officegen or pptxgenjs reader; slide-by-slide chunking
Plain text	.txt, .md	Direct read
HTML	.html, .htm	Cheerio for content extraction; strip nav/footer
Images	.jpg, .jpeg, .png	Tesseract OCR; or Haiku vision for richer extraction

22.4 Normalization Pipeline

[Raw content arrives]

│

▼

Stage 1: Content extraction (file type-specific parser)

│

▼

Stage 2: PII redaction (regex + Haiku pass)

│ ├── Zero-tolerance: SSN, credit card → quarantine + alert (aggregation per §22.4a below)

│ └── Tolerable: names, emails, phones → redact with [REDACTED]

▼

Stage 3: AI normalization (Haiku call)

│ ├── Suggest category

│ ├── Extract cruise line / ship / destination

│ ├── Generate summary

│ ├── Score quality (0-1)

│ ├── Score authority (0-1)

│ ├── Detect pricing

│ ├── Detect promo dates

│ └── Score relevance for global scope (0-1)

▼

Stage 4: Auto-flag for global review if score_global ≥ 0.6

│

▼

Stage 5: Insert into knowledge_ingestion_queue
(status='ready_for_review')

│

▼

[Tenant admin reviews]

│

▼

Tenant approves → promote to knowledge_chunks (scope='tenant')

│

▼ (if auto-flagged or platform admin manually promotes)

Platform admin reviews global queue

│

▼

Platform admin promotes → scope='global' (in addition to tenant scope)

22.4a Quarantine Alert Aggregation

The Stage 2 zero-tolerance PII quarantine alerts can burn out platform admin if a tenant repeatedly submits content with PII. Aggregation rules:

First instance per tenant: alert immediately

The first zero-tolerance match from a tenant produces an immediate alert. Content remains quarantined. Platform admin reviews.

Subsequent instances within 24 hours: aggregate

Subsequent matches from the same tenant within a 24-hour window do NOT produce individual alerts. Instead, they're aggregated into a single rolling "additional matches" summary that updates the original alert:

- Total count of additional matches in the window.
- Sample (first 3) of the quarantined content categories (NOT the content itself).
- Time range of the matches.

The original alert is updated in-place; no new alert email/page.

After 24 hours

The aggregation window resets. The next match from this tenant produces a fresh alert (which itself may be the start of a new aggregation window).

Pattern detection feeds tenant abuse signal

If a tenant produces aggregated alerts on 3+ consecutive days, the pattern indicates either (a) a tenant systematically submitting PII-bearing content, or (b) a tenant whose content sources contain PII the tenant isn't aware of. Either way, the pattern triggers a §27 abuse signal of type `rag_pii_recurring` which:

- Surfaces to platform admin in the §27.10 dashboard.
- Optionally throttles the tenant's RAG submission rate.
- Triggers an outreach email to the tenant explaining the pattern.

Schema

```
        ALTER TABLE public.tenants
          ADD COLUMN pii_quarantine_alert_window_start TIMESTAMPTZ,
          ADD COLUMN pii_quarantine_alert_count_in_window INTEGER NOT NULL
DEFAULT 0,
          ADD COLUMN pii_quarantine_recurring_days INTEGER NOT NULL DEFAULT
0;
```

The Inngest function `pii-quarantine-aggregate` updates these columns on each quarantine event and emits aggregated alerts.

Calls Worth Flagging

- **The first alert is always immediate.** Aggregation kicks in only for repeated matches.
- **Aggregation window is 24 hours, NOT calendar day.**
- **3-day recurring threshold for abuse signal is conservative.** A tenant having one bad day shouldn't be throttled.
- **Quarantined content remains quarantined regardless of aggregation.** The rule affects ALERT delivery only.

22.5 Tenant Review Queue

- List view of pending items (filter: by source type, by category, by date)
- Side-by-side: original content + AI-normalized version + suggested metadata
- Edit affordances: content, category, authority override (with reason), expiry date
- Bulk approve (with safety prompt for >10 items)

- Reject with reason (feeds tenant-specific abuse signals per §27)

22.6 Revised Global Review Model

DESIGN DECISION: Earlier drafts required tenant permission before any tenant chunk could be promoted to global. Revised: platform admin can promote ANY approved tenant chunk to global scope. Tenant retains use of the chunk (it remains in their scope too); platform gets license via ToU. Tenant cannot block promotion.

Rationale: tenants benefit from shared knowledge collectively. A gatekeeping right by individual tenants creates fragmentation. ToU acknowledgment grants platform the license.

Survives termination: The license granted by ToU acceptance is perpetual and irrevocable. Globally-promoted chunks survive tenant termination per §15.14.3. Post-termination review handling for chunks whose origin tenant is gone is specified in §15.14.4. The ToU and ICA language requirements (§15.14.6) make this stance explicit to tenants at signup.

22.7 Four-Tab Global Review Queue

Platform admin sees a queue with four tabs for triaging:

Tab	Source	Volume	Action
1. Auto-flagged	AI scored global-relevance ≥ 0.6 at ingestion	Moderate	Review and promote / pass
2. Tenant-promoted	Tenant clicked ‘suggest for global’	Low	Quick review; was tenant-initiated
3. All approved chunks	Any tenant chunk with relevance 0.3-0.6	Large	Proactive browsing; promote selectively
4. Likely tenant-specific	Relevance < 0.3	Largest	Search-first; not loaded by default; one-off promotion only

22.8 Promotion Mechanics

When platform admin promotes:

- Create new `knowledge_chunks` row with `scope = 'global'`. The promoted row records its `origin_tenant_id` for forensic traceability (kept even after the origin tenant terminates — see §15.14.3).
- Original tenant-scoped chunk remains intact (tenant continues to see it).
- If content is identical, both chunks reference each other via metadata (avoid double-retrieval of same content).
- Promotion is performed inside `withPlatformAdminAudit` per §5.4.8 with `reason = 'rag_chunk_promotion_to_global'`. The audit record captures the admin user, the chunk IDs (original and promoted),

and the platform admin's review notes.
- Reversible via demote action. Demote moves the global chunk back to tenant scope (or hard-deletes it for clear violations). Demote is also a `withPlatformAdminAudit` operation with `reason = 'rag_chunk_demotion'`.

If the origin tenant is terminated at the time of demote:

- Demote-to-tenant-scope behavior: the chunk moves back to the terminated tenant's scope and is subject to the 90-day deletion per §25.2. Effectively a delayed deletion.
- Demote-to-hard-delete behavior: chunk is removed immediately from the corpus.

The post-termination review queue (§15.14.4) is the primary surface for these decisions when the origin tenant has terminated.

22.9 Suggested Browser Extension Capability

- Floating button on supported sites (cruise line domains, partner sites)
- Right-click → 'Add to knowledge base'
- Captures: URL, page title, selected text (or full page on demand)
- Tags page automatically with detected entities
- OAuth-based auth to platform; per-user, not per-extension-instance

22.10 iOS Shortcut Capability

- Installable via share-sheet shortcut link
- Accepts: text, URL, image, PDF
- Calls platform API with attachment + optional user note
- OAuth-based auth; token stored in Shortcuts app

22.11 Quality Signals from Submissions

- AI quality score (Haiku-derived) — early filter
- Tenant approval rate (% approved vs rejected) — feeds reputation
- Customer feedback on responses citing chunks — per §6.10 authority loop
- Duplicate detection rate — flagged for review patterns

22.12 Re-Ingestion

Tenant may re-submit updated content (e.g., a refreshed promo PDF). Pipeline detects duplicates via content_hash and offers:

- Replace existing chunk (preserves chunk ID, updates content)

- Add as new chunk with supersedes link
- Cancel re-submission

22.13 Failure Modes

Failure	Behavior
File parse error	Item marked failed; tenant sees error message; can retry or submit text manually
AI normalization timeout	Retry with backoff; if persistent, queue for manual review without AI metadata
PII zero-tolerance trigger	Quarantine; alert platform admin; tenant sees friendly message
Duplicate detected	Surface for tenant decision (replace / add / cancel)
File too large	Reject at upload boundary; suggest splitting

Section 23 — Email & Notifications

Transactional and marketing email infrastructure. Includes the pre-cruise email series, which combines practical operational content with destination excitement to drive customer engagement and reviews.

23.1 Provider Architecture

- Outbound: Resend (preferred for low operational cost, simple React Email integration)
- Templates: React Email components, server-rendered
- Inbound (optional, tenant-by-tenant): Gmail API via Google Pub/Sub
- SMS (Phase 3): Twilio, deferred from v6

23.2 Email Log

CREATE TABLE public.email_log (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

tenant_id UUID NOT NULL REFERENCES public.tenants(id),

user_id UUID REFERENCES public.users(id),

contact_id UUID REFERENCES public.contacts(id),

to_email TEXT NOT NULL,

from_email TEXT NOT NULL,

```
reply_to TEXT,
subject TEXT NOT NULL,
template_id TEXT NOT NULL,
template_version INTEGER,
template_variables JSONB,
– Provider
resend_message_id TEXT UNIQUE,
– Status
status TEXT NOT NULL CHECK (status IN (
'queued','sent','delivered','soft_bounced','hard_bounced',
'complained','rejected','suppressed'
)),
sent_at TIMESTAMPTZ,
delivered_at TIMESTAMPTZ,
bounced_at TIMESTAMPTZ,
complained_at TIMESTAMPTZ,
bounce_reason TEXT,
– Categorization
email_category TEXT, –
'transactional','marketing','pre_cruise','group_invitation'
related_booking_id UUID REFERENCES public.bookings(id),
related_group_id UUID REFERENCES public.group_bookings(id),
– Audit
created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE TABLE public.email_suppressions (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES public.tenants(id),
email_address TEXT NOT NULL,
reason TEXT NOT NULL CHECK (reason IN (
'hard_bounce','complaint','unsubscribe_all',
'unsubscribe_marketing','unsubscribe_travel_news'
)),
suppressed_at TIMESTAMPTZ DEFAULT NOW(),
```

expires_at TIMESTAMPTZ,

UNIQUE (tenant_id, email_address, reason)

);

23.3 CAN-SPAM Compliance

- Physical mailing address in every email footer (from tenants.mailing_address)
- Unsubscribe link in every marketing email
- ‘About this email’ link to preferences
- Hard bounces and complaints auto-suppressed within tenant scope
- Bulk emails identify themselves as such
- From address is tenant’s identity (per §16.4 email pattern)

23.4 Pre-Cruise Email Series

Four pre-sailing emails enrich the customer experience. Each combines practical info with excitement-building content.

Timing	Theme	Practical	Excitement Component
T-90 days	Anticipation begins	Documentation reminder; passport check; pre-cruise to-dos	Destination teasers; must-do experiences per port; traveler stories; did-you-know facts; suggested reads/videos
T-30 days	Final prep window	Reservation reminders for excursions, dining, spa; check-in window; final payment	Itinerary visualization; personalized recommendations per CRM; specialty experiences; pack inspiration; social proof from prior travelers
T-7 days	Almost there	AI-generated packing checklist; deck plans; ship highlights; cruise-line-specific tips	Embarkation visuals; first-day-aboard inspiration; what to expect at port
		Carry-on essentials callout (passport, paperwork,	Tomorrow’s first port preview;

T-1 day	Tomorrow!	medications); port directions via RAG-stored official URLs; weather for all stops	what to expect day-of

Carry-On Essentials Callout (T-1 day)

CRITICAL CONTENT:** **The T-1 day email MUST include a prominent callout. Passports, travel paperwork, and medications go in CARRY-ON, not checked luggage. This single piece of advice prevents the most common avoidable trip-ruining mistakes.

Port Information

T-1 day email shows directions to embarkation port from RAG-stored official port authority URLs (not Google Maps — platform doesn't know customer's starting point and won't guess).

Required platform launch task: seed RAG with port chunks for major North American departure ports (Miami, Port Canaveral, Galveston, Seattle, Vancouver, New York, Boston, Baltimore, New Orleans, Los Angeles, San Diego, San Francisco, Long Beach, Tampa, Jacksonville, Mobile, Norfolk). Each port chunk includes: official URL, terminal addresses, parking info, transit/Uber drop-off points, general arrival advice.

Caching

Per-customer pre-cruise content is generated once and cached. Estimated cost per send: $0.01-0.02 in AI inference. Schema:

CREATE TABLE public.pre_cruise_email_content (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

tenant_id UUID NOT NULL REFERENCES public.tenants(id),

booking_id UUID NOT NULL REFERENCES public.bookings(id),

contact_id UUID NOT NULL REFERENCES public.contacts(id),

email_phase TEXT NOT NULL CHECK (email_phase IN ('t_90','t_30','t_7','t_1')),

generated_content JSONB NOT NULL, – body sections, image refs, links

companion_page_url TEXT, – optional web companion for richer content

generated_at TIMESTAMPTZ DEFAULT NOW(),

sent_at TIMESTAMPTZ,

UNIQUE (booking_id, email_phase)

);

23.5 Companion Web Pages

Pre-cruise emails link to companion web pages (under tenant's domain) with richer content: image galleries, embedded videos, detailed itineraries. Companion pages are token-gated like group invitations; no auth required.

23.6 Email Categories & Rate Limits

Category	Examples	Rate Limit
Transactional	Booking confirmation, quote sent, password reset	Unlimited (must-send)
Pre-cruise	T-90, T-30, T-7, T-1 series	Bound to schedule; not subject to general rate limits
Group invitation	Initial invitation + reminders per §18.8	3 emails / 24h per invitee
Marketing	Travel news, promotions	Strict opt-in; 4 / month max per contact
Travel news	Curated content opt-in	Weekly digest if subscribed

23.7 Bounce & Complaint Handling

Email sent

Resend webhook fires:

├──► delivered → log only

├──► soft bounce → retry up to 3 times over 24h

├──► hard bounce → suppress + flag email_status='invalid'

├──► complaint → suppress + flag email_status='complained'

└──► open / click → log for engagement metrics (no PII)

23.8 In-App Notifications

```
CREATE TABLE public.notifications (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES public.tenants(id),
user_id UUID NOT NULL REFERENCES public.users(id),
category TEXT NOT NULL,
title TEXT NOT NULL,
```

body TEXT,

link_url TEXT,

icon TEXT,

read_at TIMESTAMPTZ,

dismissed_at TIMESTAMPTZ,

created_at TIMESTAMPTZ DEFAULT NOW()

);

CREATE INDEX notifications_user_unread_idx

ON public.notifications(user_id, read_at) WHERE read_at IS NULL;

Bell icon in app header shows unread count. Categories: booking_update, commission_settled, group_activity, escalation, system.

23.9 Gmail Inbound (Optional)

- Tenant connects their Gmail/Workspace via OAuth
- Google Pub/Sub topic notifies on new messages
- Platform fetches metadata, matches to existing contacts/conversations
- AI summarizes and routes; may surface in CRM activity timeline
- Auto-reply optional (off by default)

Section 24 — Chat UI

The customer-facing chat interface. Design priorities: clarity, AI disclosure, accessibility, and rapport-building tone matching.

24.1 Layout

Desktop

┌──┐

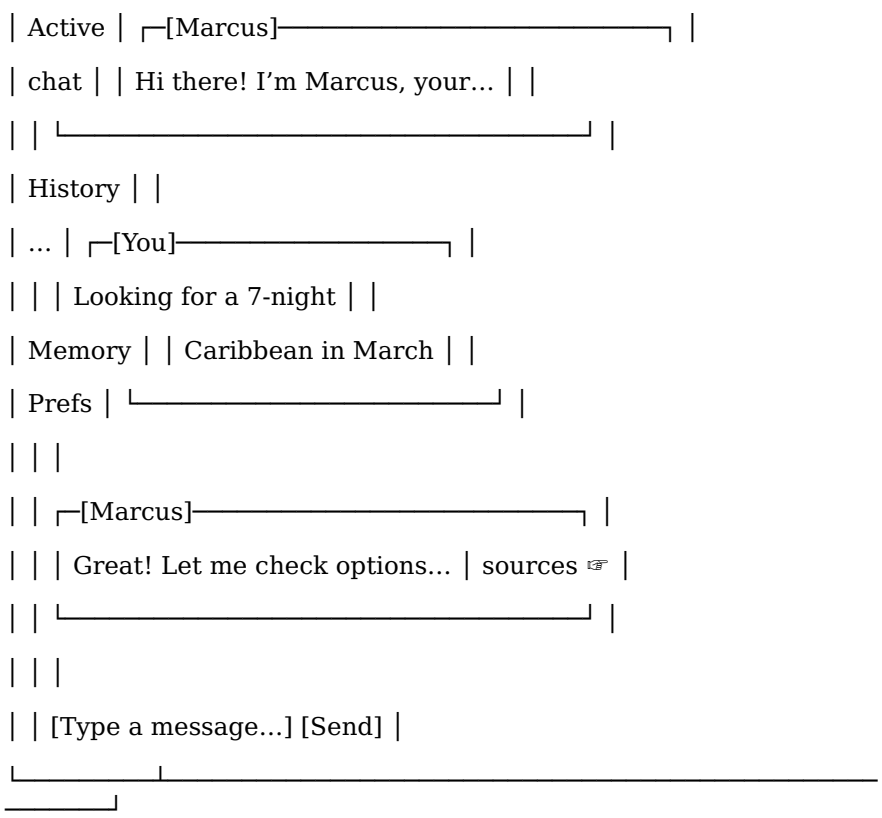
│ [Tenant Logo] [Tenant Name] [Sign In / Account] │

├──┤

│ AI-assisted chat — your conversations are reviewed for quality │ ← persistent disclosure

├──────────┬───┤

│ │ │



Mobile

- Single-pane stack; menu drawer for history/memory/prefs
- Chat takes full width; messages flow vertically
- Persona avatar small (left side of assistant message bubble)
- Sticky compose bar at bottom; sticky disclosure at top

24.2 Persistent AI Disclosure

Visible at top of every chat: 'AI-assisted chat — your conversations are reviewed for quality'. State law compliance per §17.6 satisfied by the AI Liability Disclaimer acceptance; the persistent banner provides ongoing reminder.

24.3 Streaming with Auto-Scroll

Messages stream token-by-token. Auto-scroll behavior respects user intent:

- If user is at bottom: keep scrolling as response streams
- If user has scrolled up: don't yank them down (they're reading earlier content); show 'New message' indicator instead
- Click 'New message' indicator → smooth scroll to latest

24.4 Message Components

Element	Behavior
Persona avatar	Shown on assistant messages; from tenant override or platform default
Persona name	Tenant override if applicable; else platform default
Sources indicator	If RAG used: subtle icon, click to expand
Timestamp	Hover shows full; UI shows relative (‘2 min ago’)
Feedback buttons	Thumbs up / thumbs down on assistant messages; reason optional on down
Copy button	Available on hover; copies plain text

24.5 Tone Matching & Rapport Building

Five-Level Scale

Level	Style	Example
1 — Formal	Professional, courteous, full sentences	I would be delighted to help you explore Caribbean cruise options.
2 — Professional warm	Default for most contexts	Happy to help you find the right Caribbean cruise.
3 — Casual	Conversational, contractions, lighter rhythm	Sure! Let’s find a great Caribbean cruise for you.
4 — Friendly informal	First-name basis, lighter language, mild colloquialism	Awesome — Caribbean cruises are my jam. What’s the occasion?
5 — Matched energy	Mirroring customer’s slang and rhythm; warmth without sycophancy	Heck yes, let’s get you on a Caribbean cruise. What’re we thinking?

Configuration

- Default tone level: 2 (professional warm)
- Per-tenant cap (persona_tone_max_level): default 3 (casual); tenant can raise to 5
- All five levels open at launch — not phased; safeguards (supervisor tone_drift check, profanity opt-in) make this safe
- Profanity allow: opt-in per tenant (default FALSE); does not auto-enable at any level

- Slang allow: separate flag from profanity; default TRUE at level 4+

Rapport Signal Sources

AI infers an appropriate level for each customer from signals:

- Customer's own message tone (casual vs formal)
- Stated preferences ('call me Mike, not Michael')
- Conversation history with tenant
- Persona-base inclination (Marcus more casual; Priya more polished)

Initial level: midpoint between customer's apparent style and tenant default. Adjusts upward gradually within the tenant's cap.

Topic-Aware Modulation

Serious topics override tone level downward:

Topic	Treatment
Medical / accessibility concern	Stays at level 1-2 regardless
Financial discussion (refunds, deposits, large sums)	Drops to level 2-3
Cancellation, complaint, escalation	Drops to level 1-2; empathy increased
Booking confirmation, contract terms	Drops to level 2
Casual exploration, trip ideas	Free to be at customer's preferred level

Customer Override

Customer can request tone change explicitly:

- 'Be more casual' / 'Lighten up' → bump level up by 1 (within cap)
- 'Be more professional' / 'Tone it down' → drop level by 1
- 'Cut the small talk' → drop to level 2; stay direct
- Customer override persisted in customer_memories.rapport_tone_level

Supervisor tone_drift Check

The `tone_drift` check has two layers: a **heuristic layer** that flags tone-vs-context mismatch, and a **deterministic lexical layer** that blocks specific terms regardless of tone configuration. Both run on every AI response candidate before it surfaces to the customer.

Heuristic layer

- Detects responses far above tenant's `persona_tone_max_level`
- Detects profanity when `allow_profanity = FALSE`
- Detects mismatched seriousness (e.g., joking while customer is

upset)
- Detects sycophancy that reads as fake (independent of tone level)
- Flags severe drift; regenerates with stricter tone instruction

Deterministic lexical layer (slurs and hate speech)

A curated deny-list of slurs and hate-speech terms is matched against every AI response candidate. Matches are blocked regardless of:

- Tenant's `persona_tone_max_level` configuration
- Tenant's `allow_profanity` setting
- Customer's message content (even if the customer used the term, the AI does not mirror or reciprocate)
- Persona-level overrides

On match:

- Response is rejected and regenerated with explicit instruction: "Your previous response contained language we don't allow. Please rewrite without [specific term redacted]. Keep the same intent and tone otherwise."
- The regeneration counts toward the per-conversation regen budget per §10.1a.
- If three consecutive regenerations all match the lexical filter, the conversation auto-escalates to a human and the platform admin is alerted (the persona appears to be stuck producing matched content — could indicate prompt injection or persona corruption).
- Every match is logged to `audit_log` with the matched term hash (NOT the term itself).

Why deterministic, not heuristic, for this category

Slurs and hate speech are different from "tone too casual" or "profanity in a low-tone context." They are categorical: the term either appears or it doesn't. A heuristic check (LLM judging "is this slur-adjacent?") produces false negatives that screenshot badly. A deterministic check produces zero false negatives on listed terms.

The deny-list

The list itself is maintained in `src/lib/supervisor/hate-speech-denylist.ts` (path indicative). The list:

- Includes slurs targeting race, ethnicity, religion, gender, sexuality, disability, and other protected categories.
- Includes hate-speech terms and known dog-whistles.
- Is reviewed and updated quarterly by platform admin per §26.11.
- Updates write to `audit_log` with `action = 'denylist.updated'` and a count of additions/removals (NOT the specific terms).
- Includes context-aware exclusions (terms that are slurs in one context but anatomical/culinary in another).

The deny-list is platform-wide; tenants cannot add or remove from it.

Tenant-side additional filtering

A tenant may add additional terms to a **per-tenant supplemental deny-list** via tenant admin console (Pro+ tier). This adds to the platform list, does not subtract from it.

Customer-side matching is informational

The lexical filter runs on AI output. The platform does NOT block customers from typing slurs or hate-speech themselves. The AI simply doesn't reciprocate. If a customer's message contains slurs targeted at the AI or the tenant, the supervisor may suggest a topic-level escalation per §10.3 — the customer's behavior is the tenant's call to handle, not the platform's.

Calls Worth Flagging (tone_drift)

- **Profanity remains allowed (and mirrorable at tone level 5 when opted in).** This is the existing design and not changed. The new layer is specifically for slurs and hate speech.
- **The deny-list is a maintenance burden.** Quarterly review is the cadence. Without active maintenance, the list ages out.
- **Three consecutive regen-on-match triggers human escalation.** This is the prompt-injection detector.
- **The platform list is platform-wide; tenant supplemental list is additive only.** Tenants cannot weaken the platform's filtering.
- **Audit log records matches by hash, not by term.** Protects the deny-list from being reverse-engineered.

24.6 Persona Switching In-Conversation

When AI suggests handoff (per §9.8), customer sees:

I think Maya (our accessibility specialist) would be the right person for these questions. Want me to bring her in? [Yes, switch to Maya] [No, stick with Marcus]

On Yes: active_persona_id updates; Maya's first message includes brief context summary.

24.7 Message Persistence

Messages saved as user types (draft) and on submit (final). Conversation reconnect after network drop reloads state from server.

24.8 Anonymous Chat Limit

Anonymous chat is the highest abuse-surface chat endpoint on the platform: no tenant attribution (every Anthropic call comes off the platform's overall spend), no identity check, no recoverable abuse signal beyond what the platform itself collects. The rate limit is enforced at multiple identifiers, not just session.

Three-identifier limit (whichever is most restrictive)

Anonymous chat is rate-limited per:

1. **Session.** Default: 5 messages per session.
2. **IP address.** Default: 15 messages per IP per rolling 24 hours.
3. **Client fingerprint.** Default: 10 messages per fingerprint per rolling 24 hours. Fingerprint is a server-derived hash of (user-agent, accept-language, screen size if available, canvas fingerprint

if available) — best-effort, not cryptographic.

The most restrictive of the three applies. A user clearing cookies still hits IP and fingerprint limits. A user switching browsers still hits IP. A user switching networks still hits fingerprint.

Behavior at limit

When any identifier hits its limit:

- The signup wall is shown (per existing §24.8 behavior).
- The wall message indicates the limit was reached but does NOT specify which identifier (avoiding feedback that helps adversaries optimize).
- The customer is invited to sign up. Once signed up, anonymous limits no longer apply — the authenticated customer limit per §24.9 (below) applies instead.

Tenant abuse signal feeding

When an IP or fingerprint repeatedly hits limits across multiple sessions (3+ sessions from the same IP all hitting the 5-message limit within 24h), the platform treats this as a signal that the IP/fingerprint is being used adversarially. The tenant whose subdomain was hit is alerted via §27 abuse monitoring with a new signal type `anonymous_chat_burst`.

The signal contributes to §27's abuse-monitoring thresholds at the *platform admin alert* level only — it does NOT directly throttle the tenant.

Tier-aware ceiling

The defaults above are platform-wide. A tenant under active abuse signal may have their anonymous chat limit reduced by platform admin:

- Normal tenant: 5 / 15 / 10
- Under abuse signal: 2 / 5 / 3 (configurable downward from platform admin console)

Schema

```
        CREATE TABLE public.anonymous_chat_counters (
          identifier_type TEXT NOT NULL CHECK (identifier_type IN
('session', 'ip', 'fingerprint')),
          identifier_value TEXT NOT NULL,
          tenant_id UUID NOT NULL REFERENCES public.tenants(id),
          current_count INTEGER NOT NULL DEFAULT 0,
          first_message_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
          last_message_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
          limit_hit_at TIMESTAMPTZ,
          PRIMARY KEY (identifier_type, identifier_value, tenant_id)
        );

        CREATE INDEX anonymous_chat_counters_cleanup_idx
          ON public.anonymous_chat_counters(last_message_at);

        -- Cleanup cron: rows older than 7 days hard-deleted nightly.
```

Configurable knobs (in `platform_settings`)

Setting	Default	Description
`anon_chat_limit_per_session`	5	Per-session message cap
`anon_chat_limit_per_ip_24h`	15	Per-IP 24-hour rolling cap
`anon_chat_limit_per_fingerprint_24h`	10	Per-fingerprint 24-hour rolling cap
`anon_chat_limit_per_session_under_abuse`	2	Reduced session cap when tenant under abuse signal
`anon_chat_limit_per_ip_under_abuse`	5	Reduced IP cap under abuse
`anon_chat_limit_per_fingerprint_under_abuse`	3	Reduced fingerprint cap under abuse

Platform-wide only. No per-tenant configurability for anonymous chat.

Anonymous sessions are preserved for transfer per §11.6.

Calls Worth Flagging

- **Fingerprinting is best-effort, not a security boundary.** Fingerprint defeats casual evasion; sophisticated adversaries hit other layers.
- **The cleanup cron is required.** Without it, counter rows accumulate indefinitely.
- **Privacy considerations:** IP addresses are PII in some jurisdictions (GDPR). 7-day retention bounds exposure.
- **The “don’t specify which limit was hit” rule is deliberate.** Telling a user which limit they hit defeats the layered defense.

24.9 Authenticated Customer Chat Rate Limit

Purpose

This rate limit protects against authenticated customers who use the AI as a free travel research chatbot without booking through the tenant. It is per-(customer, tenant) and operates on a rolling 30-day window. Same shape as §27’s three-tier abuse monitoring structure, but at the customer level rather than the tenant level.

The rate limit does NOT apply to:

- Anonymous customer chat — covered by §24.8.

- Tenant users (agents, coordinators, owner accounts).
- Platform admin users.

Three-tier structure

Tier	Default messages in 30d	Behavior
Soft1	20	Persona delivers gentle in-character nudge to focus on what cruise the customer would like to book. No system signal; nudge embedded in regular AI response.
Soft2	30	Persona delivers more urgent in-character warning about reduced future availability. Still embedded in AI response.
Hard	40	Chat blocked. Customer sees a system-rendered message (clearly platform-spoken, not in-character) explaining the cap and offering escalation. Essential operations (viewing existing bookings, account settings) and human handoff remain available. Platform admin alerted with conversation summary.

The counter is the running count of customer-sent messages in the trailing 30 days. AI-sent messages don’t count against the cap.

Booking-based capacity bonus

A customer with **any future-dated booking in the tenant** receives a percentage bonus applied to all three thresholds. The bonus is expressed as a percentage (default +100%, meaning effective ×2 capacity). Defaults under +100% bonus: 40 / 60 / 80.

“Future-dated booking” means: a `bookings` row in this tenant with `sailing_date >= NOW()` AND `status NOT IN ('cancelled', 'no_show', 'refunded')`.

The bonus is computed lazily on each message — there’s no stored “bonus_active” flag. When the last future-dated booking sails (or gets cancelled), the bonus silently drops back to 0%.

Scope: per-(customer, tenant)

The 30-day counter is scoped per `(user_id, tenant_id)`. A customer signed in to multiple tenants gets a full quota in each. A booking in tenant A applies the bonus to caps in tenant A only.

Tier-aware configurability

All four configurable values (Soft1, Soft2, Hard thresholds, plus booking bonus percentage) have platform defaults set in §5.5 `platform_settings`. Pro+ tier tenants can override these values in their tenant admin console; lower tiers use platform defaults.

Tenant overrides are constrained:

Constraint	Reason
Hard limit cannot exceed `platform_settings.customer_chat_limit_hard_ceiling` (default 200)	Prevents tenants from setting absurdly high caps
Hard limit cannot be lower than `platform_settings.customer_chat_limit_hard_floor` (default 15)	Prevents customer-hostile gating
Soft1 < Soft2 < Hard always (validated on save)	Sanity
Booking bonus percentage between 0% and 400%	Generous range

Changes take effect at the next message.

Customer-facing experience

At Soft1 (in-character nudge)

The persona is prompted (via prompt augmentation per §9.7) to weave a gentle focus-redirect into the response. Example:

> "I've enjoyed our chat about Mediterranean cruises! At this point we've explored quite a bit — want to narrow it down to one or two sailings you're seriously considering? I can put together a detailed quote on whichever you like best."

The supervisor's `tone_drift` check (§10.2) verifies the nudge stays in-character.

At Soft2 (in-character warning)

More urgent. Example:

> "I want to be upfront with you — at this rate, we won't be able to keep chatting much longer if we don't find a cruise that works for you. I can put together quotes on the top three sailings we've discussed, or we can switch tracks if cruising isn't quite right for this trip. What would you like to do?"

At Hard limit (system message)

When the customer's next message would push them over the hard cap, the message is NOT sent to the AI. A system message renders directly in chat:

> **Chat limit reached.**
>
> You've reached the message limit for this billing period. To continue chatting, you have two options:
>
> - **Book a cruise** — booking with us doubles your chat quota for as long as you have an upcoming trip.
> - **Talk to a human** — request a handoff to a trip consultant.
>
> Your quota resets [30 days from oldest counted message].
>
> [Talk to a human] [View my bookings]

The system message is rendered server-side and inserted directly into the chat — no AI call involved. Customer can:

- Click "Talk to a human" → standard escalation per §24.10 (existing).
- Click "View my bookings" → navigates to bookings list.
- View existing booking details, account settings, group invitations — all unaffected.
- Cannot send new messages to the AI persona.

Platform admin alert at hard limit

When a customer hits the hard limit, an alert is generated with a conversation summary:

```
        {
          "alert_type": "customer_chat_hard_limit",
          "tenant_id": "uuid",
          "user_id": "uuid",
          "user_email": "customer@example.com",
          "conversation_summary": {
            "message_count_total": 40,
            "first_message_at": "2026-04-17T...",
            "last_message_at": "2026-05-17T...",
            "topics_discussed": ["Mediterranean cruises", "Royal Caribbean
ships", "drink package options", "cabin categories"],
            "booking_intent_signal": "low | medium | high",
            "active_bookings_count": 0,
            "future_bookings_count": 0
          },
          "summary_generated_at": "2026-05-17T..."
        }
```

The `conversation_summary` is generated by a Haiku call. Platform admin can review and decide whether to:

- Manually raise the customer's cap (audited via `withPlatformAdminAudit` per §5.4.8 with `reason = 'customer_chat_cap_override'`).
- Reach out to the tenant about the customer.
- Take no action (most cases).

Audit

Every soft-tier crossing and every hard-limit block writes to `audit_log`:

Event	action	changes payload
Soft1 crossed	`customer_chat.soft1_nudge_issued`	message_id, persona_id, counter value
Soft2 crossed	`customer_chat.soft2_warning_issued`	same
Hard limit hit	`customer_chat.hard_limit_blocked`	counter value, summary reference
Manual cap override by admin	`customer_chat.cap_override`	old/new caps, admin user, reason

Schema

```
        -- Per-customer-per-tenant rolling counter.
        CREATE TABLE public.customer_chat_counters (
          user_id UUID NOT NULL REFERENCES public.users(id) ON DELETE
CASCADE,
          tenant_id UUID NOT NULL REFERENCES public.tenants(id) ON DELETE
RESTRICT,
          current_count INTEGER NOT NULL DEFAULT 0,
          last_message_at TIMESTAMPTZ,
          soft1_last_issued_at TIMESTAMPTZ,
          soft2_last_issued_at TIMESTAMPTZ,
          hard_limit_hit_at TIMESTAMPTZ,
          hard_limit_summary_audit_id UUID REFERENCES public.audit_log(id),
          resolved_soft1_cap INTEGER,
          resolved_soft2_cap INTEGER,
          resolved_hard_cap INTEGER,
          resolved_booking_bonus_percent INTEGER,
          resolved_at TIMESTAMPTZ,
          PRIMARY KEY (user_id, tenant_id)
        );

        -- Tenant overrides (Pro+ tier only — enforced at write).
        CREATE TABLE public.customer_chat_tenant_overrides (
          tenant_id UUID PRIMARY KEY REFERENCES public.tenants(id) ON DELETE
RESTRICT,
          soft1_cap INTEGER,
          soft2_cap INTEGER,
          hard_cap INTEGER,
          booking_bonus_percent INTEGER,
          updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
          updated_by_user_id UUID REFERENCES public.users(id)
        );
```

The cap-resolution function

```
        CREATE OR REPLACE FUNCTION public.resolve_customer_chat_caps(
          p_tenant_id UUID,
          p_user_id UUID
        ) RETURNS TABLE (
          soft1_cap INTEGER,
          soft2_cap INTEGER,
          hard_cap INTEGER,
          booking_bonus_percent INTEGER
        )
```

```
        LANGUAGE plpgsql
        STABLE
        SET search_path = ''
        AS $$
        DECLARE
          v_tier TEXT;
          v_has_future_booking BOOLEAN;
          v_base_soft1 INTEGER;
          v_base_soft2 INTEGER;
          v_base_hard INTEGER;
          v_bonus INTEGER;
        BEGIN
          -- 1. Get tenant tier (drives whether overrides apply).
          SELECT tier_definitions.name INTO v_tier
            FROM public.tenants
            JOIN public.tier_definitions ON tenants.tier_id =
tier_definitions.id
            WHERE tenants.id = p_tenant_id;

          -- 2. Check tenant override (only if Pro+).
          IF v_tier IN ('sub_host_pro', 'sub_host_agency', 'byo_agency')
THEN
            SELECT cco.soft1_cap, cco.soft2_cap, cco.hard_cap,
cco.booking_bonus_percent
              INTO v_base_soft1, v_base_soft2, v_base_hard, v_bonus
              FROM public.customer_chat_tenant_overrides cco
              WHERE cco.tenant_id = p_tenant_id;
          END IF;

          -- 3. Fall back to platform defaults.
          IF v_base_soft1 IS NULL THEN
            SELECT (value::TEXT)::INTEGER INTO v_base_soft1
              FROM public.platform_settings WHERE key =
'customer_chat_soft1_cap';
          END IF;
          IF v_base_soft2 IS NULL THEN
            SELECT (value::TEXT)::INTEGER INTO v_base_soft2
              FROM public.platform_settings WHERE key =
'customer_chat_soft2_cap';
          END IF;
          IF v_base_hard IS NULL THEN
            SELECT (value::TEXT)::INTEGER INTO v_base_hard
              FROM public.platform_settings WHERE key =
'customer_chat_hard_cap';
          END IF;
          IF v_bonus IS NULL THEN
            SELECT (value::TEXT)::INTEGER INTO v_bonus
              FROM public.platform_settings WHERE key =
'customer_chat_booking_bonus_percent';
          END IF;

          -- 4. Check for future-dated booking.
          SELECT EXISTS (
            SELECT 1 FROM public.bookings
            WHERE bookings.user_id = p_user_id
              AND bookings.tenant_id = p_tenant_id
              AND bookings.sailing_date >= NOW()
              AND bookings.status NOT IN ('cancelled', 'no_show',
'refunded')
          ) INTO v_has_future_booking;
```

```
    -- 5. Apply bonus if applicable.
    IF v_has_future_booking THEN
      soft1_cap := v_base_soft1 + (v_base_soft1 * v_bonus / 100);
      soft2_cap := v_base_soft2 + (v_base_soft2 * v_bonus / 100);
      hard_cap := v_base_hard + (v_base_hard * v_bonus / 100);
      booking_bonus_percent := v_bonus;
    ELSE
      soft1_cap := v_base_soft1;
      soft2_cap := v_base_soft2;
      hard_cap := v_base_hard;
      booking_bonus_percent := 0;
    END IF;

    RETURN NEXT;
  END;
  $$;

  REVOKE EXECUTE ON FUNCTION public.resolve_customer_chat_caps(UUID,
UUID) FROM public;
  GRANT EXECUTE ON FUNCTION public.resolve_customer_chat_caps(UUID,
UUID) TO authenticated;
```

Counter maintenance

Two paths keep `customer_chat_counters.current_count` accurate:

1. **On each message send** (sync, before AI call): increment counter for `(user_id, tenant_id)`. Check counter against resolved caps; trigger appropriate behavior.
2. **Nightly recompute** (Inngest cron `customer-chat-counter-recompute`): for each row, recount messages from the last 30 days. Safety net for drift.

Configurable knobs (in `platform_settings`)

Setting	Default	Range
`customer_chat_soft1_cap`	20	5–500
`customer_chat_soft2_cap`	30	5–500
`customer_chat_hard_cap`	40	15–200
`customer_chat_booking_bonus_percent`	100	0–400
`customer_chat_limit_hard_ceiling`	200	50–500
`customer_chat_limit_hard_floor`	15	5–50
`customer_chat_window_days`	30	7–90

Persona prompt augmentation

When the conversation handler resolves caps and finds the customer is approaching a soft tier, the persona's system prompt is augmented:

- At Soft1 (current_count >= soft1_cap, < soft2_cap, no Soft1 nudge issued in last 7 days):

  ```
  Note for this turn: this customer has been chatting actively with
  you over the past few weeks. Without seeming pushy, gently
  encourage focusing on a specific cruise to book. Suggest you'd be
  happy to put together a detailed quote on a sailing they're
  seriously considering. Keep your normal personality and warmth.
  ```

- At Soft2 (current_count >= soft2_cap, < hard_cap, no Soft2 warning issued in last 3 days):

  ```
  Note for this turn: this customer has had extensive chat time
  with you and hasn't booked yet. Be more direct (still warm and
  in-character) about the conversation track. Mention that without
  finding a cruise that works, you won't be able to keep chatting
  indefinitely, and offer specific next steps.
  ```

The exact text is itself configurable via `platform_settings` keys `customer_chat_soft1_persona_prompt` and `customer_chat_soft2_persona_prompt`.

Integration with §27 abuse monitoring

Hitting customer chat limits is NOT an abuse signal against the tenant. §27 catches tenant-level patterns (bot armies); this catches per-customer behavior (one person using AI as a free research chatbot). A customer hitting Soft1/Soft2/Hard does NOT count against the tenant's §27 thresholds.

Calls Worth Flagging

- **The booking bonus is generous and configurable.** +100% default doubles the cap. Tenants can configure 0%–400%.
- **The bonus is computed lazily, not persisted as a flag.** A customer with a booking that just sailed automatically falls back to base cap.
- **Per-tenant scope means a customer can game the system by signing up across tenants.** Acceptable: each tenant pays for AI cost in their tenant.
- **The Haiku-generated summary at hard-limit hit is the human-readable signal.** Platform admin sees actionable info.
- **Soft tiers have cooldowns** (7 days for Soft1, 3 days for Soft2) to avoid nagging.
- **The hard-limit message DOES NOT use the persona.** It's clearly platform-spoken.
- **`status NOT IN ('cancelled', 'no_show', 'refunded')` for the booking check is important.** A customer who books then cancels should NOT keep the bonus.

24.10 Escalation Surface

Customer can request human at any time via:

- Explicit ask ('I'd like to talk to a person')
- Escalation button in conversation menu
- Implicit signals (frustration patterns; supervisor detects)

On escalation: topic-level escalation created (§10.3); customer sees friendly transition message; tenant agent receives notification.

24.11 Memory Indicators

Subtle UI cues when AI uses memory:

Welcome back, Sarah! Last time you mentioned you were thinking about Alaska — should we pick up there?